

[65] Quantifying TPC-H Choke Points and Their Optimizations

요약

본 논문은 TPC-H 벤치마크의 쿼리 성능을 분석하여 각 쿼리에서의 병목(choke point)을 정량화하고 최적화 기법의 효과를 평가한다. 2020년 VLDB(매우 영향력 있는 데이터베이스 학회)에 발표된 Dreseler 등의 연구는 TPC-H가 어떤 쿼리에서는 효과적 벤치마크이고, 어떤 쿼리에서는 한계가 있는지를 체계적으로 분석한다.

TPC-H 벤치마크의 개요:

- 22개의 분석용 SQL 쿼리
- 1GB에서 수 TB 규모의 데이터
- 관계형 데이터베이스의 분석 워크로드 대표
- 가장 널리 사용되는 상용 DB 벤치마크

병목(Choke Point) 분석의 중요성:

- 특정 쿼리는 조인 최적화 부족을 반영 (Q17)
- 특정 쿼리는 집계 성능을 반영 (Q1)
- 특정 쿼리는 인덱싱 기법을 반영 (Q19)
- 어떤 쿼리들은 현대 시스템에서 너무 쉬움

주요 발견:

- 각 쿼리마다 서로 다른 최적화 기법이 필요
- 기존 시스템의 최적화가 특정 패턴에만 효과적
- 벡터 검색 같은 새로운 워크로드 추가 시 기준 재설정 필요

Exqutor가 TPC-H를 벡터 검색으로 확장할 때, 이 분석은 어떤 쿼리가 벡터 부분을 추가할 가치가 있는지, 어떻게 측정할 것인지에 대한 기초를 제공한다.

상세분석

65.1 TPC-H의 22개 쿼리 분류

1.1 각 쿼리의 주요 특징

Q1: 집계 중심 쿼리

- 단일 테이블(LINEITEM)에서 대량의 행 필터링과 집계
- 스캔, 필터, 그룹화 최적화 측정
- 현대 DBMS에서는 매우 효율적

Q17, Q20: 조인 최적화

- 복잡한 조인 구조
- 조인 순서, 조인 방식 최적화가 성능에 직결
- 쿼리 최적화기의 핵심 능력 측정

Q5, Q10: 윈도우 함수와 집계

- 여러 테이블 조인 후 집계
- I/O 비용과 메모리 사용의 균형

1.2 병목의 정의

Dreseler 논문에서 제시하는 병목:

성능 병목 = 전체 실행 시간의 20% 이상 차지하는 연산

예:

- Q1: 전체 시간의 70% = SCAN + GROUP BY → 명확한 병목
- Q3: JOIN이 30%, SORT가 25%, SCAN이 20% → 다중 병목
- Q8: 모든 연산이 10% 미만 → 구체적 병목 없음

65.2 병목 분석 결과

2.1 스캔 병목

테이블 스캔이 주요 병목인 쿼리:

- Q1, Q6: 대량 데이터 스캔
- 해결책: 인덱싱, 파티셔닝, 컬럼 스토어
- 현대 DBMS의 최적화로 비교적 잘 해결됨

2.2 조인 병목

중첩 루프(Nested Loop) 조인:

- 가장 간단하지만 느림 $O(n*m)$
- 작은 데이터셋에는 괜찮음

해시 조인(Hash Join):

- 메모리가 있으면 매우 효율적 $O(n+m)$
- 메모리 부족하면 극적으로 느려짐

정렬 병합(Sort-Merge):

- 이미 정렬된 입력에 효율적
- 정렬 비용이 크면 비효율적

Q5의 경우:

- 4개 테이블의 조인
- 조인 순서가 성능에 10배 이상 영향
- 일부 DBMS는 좋은 순서를 찾지 못함

2.3 집계 병목

작은 그룹 수:

- 해시 집계 매우 효율적
- GROUP BY 병목 아님

큰 그룹 수:

- 메모리 오버플로우 위험
- 정렬 기반 집계 필요
- 디스크 I/O 증가

Q1에서 명확한 병목:

- 날짜 범위별 그룹화
- 그룹 수는 작음 (수백개)
- 실제 병목: 수억 개 행의 스캔과 필터

2.4 정렬 병목

메모리 내 정렬:

- 매우 빠름

외부 정렬(External Sort):

- 메모리 초과 시 발생
- 다중 pass로 인한 디스크 I/O
- 성능 급격히 저하

Q3, Q10 같은 쿼리에서:

- ORDER BY 절이 명시적 병목
- 중간 결과 크기가 메모리 한계 초과

65.3 최적화 기법의 효과 측정

3.1 인덱싱의 효과

인덱스 없음:

- Q6 실행시간: 1000ms
- 전체 LINEITEM 테이블 스캔 필요

인덱스 있음 (L_SHIPDATE):

- Q6 실행시간: 50ms
- 20배 성능 개선

제약:

- 모든 쿼리가 같은 인덱스로 이득 받지는 않음
- 인덱스 유지 비용 (INSERT/DELETE/UPDATE 느려짐)

3.2 파티셔닝의 효과

날짜별 파티셔닝:

- LINEITEM을 연도별로 분할
- 범위 검색 쿼리에서 스캔량 감소
- Q1 같은 쿼리에서 성능 향상

제약:

- 복잡한 조인 쿼리에서는 이득 적을 수 있음
- 파티션 프루닝(pruning) 기능 필요

3.3 컬럼 스토어의 효과

전통 행 스토어 (Row Store):

테이블:

ID	Name	Price	Date
1	Apple	\$1	2020
2	Banana	\$0.5	2020

모든 필터링에도 전체 행 디코딩

컬럼 스토어 (Column Store):

Price 컬럼만 필요하면:

Price: [1, 0.5, ...]

Date: [2020, 2020, ...]

필요한 컬럼만 로드

결과: Q1 같은 단순 쿼리에서 10배 이상 성능 향상

65.4 시스템별 성능 차이

4.1 상용 DBMS vs 오픈소스

상용 DBMS (Oracle, SQL Server):

- 복잡한 조인 최적화 우수
- 벡터화 실행 등 최신 기법 적용
- Q17 같은 복잡한 쿼리에서 강함

오픈소스 (PostgreSQL):

- 간단한 쿼리에서는 경쟁력 있음
- 복잡한 쿼리 최적화는 뒤떨어짐
- 하지만 지속적으로 개선 중

4.2 데이터 크기별 성능 차이**1GB 스케일:**

- 모든 데이터가 메모리에 올라옴
- 인덱싱, 파티셔닝의 효과 적음
- 대부분의 시스템 비슷한 성능

100GB 스케일:

- 메모리 관리가 중요해짐
- 시스템 간 성능 차이 증대
- 조인 순서 최적화의 중요성 증가

1TB 이상:

- 디스크 I/O가 절대적 병목
- 인덱싱, 파티셔닝 매우 중요
- 시스템 간 성능 격차 극대화

65.5 TPC-H 벤치마크의 현대적 한계**5.1 스키마의 단순성****현대 데이터 특성:**

- 비정규화된 스키마 (NoSQL의 영향)
- 복잡한 데이터 타입 (JSON, 배열, 맵)
- 다양한 정렬도(cardinality) 분포

TPC-H의 단순함:

- 정규화된 스키마만 사용
- 기본 데이터 타입만
- 비교적 균등한 데이터 분포

5.2 쿼리 복잡도**TPC-H의 쿼리들:**

- 대부분 10개 이하의 조인
- 명확한 WHERE 조건

현대 분석 워크로드:

- 20개 이상의 조인 가능 (DataLake)
- 중첩 쿼리, CTEs의 복잡한 사용
- 동적 쿼리 생성

5.3 벡터 검색 부재

TPC-H의 한계:

- 정확한 매칭만 가능
- 유사도 기반 검색 없음
- 의미론적 분석 불가능

현대 요구사항:

- 고객 리뷰와 텍스트 유사도 검색
- 이미지 유사도 기반 추천
- 의미론적 검색과 정확한 검색의 결합

65.6 본 논문과의 관계

Exqutor의 TPC-H 벡터 확장에서:

- **기존 병목 이해**: 벡터 검색 추가 전 TPC-H의 기존 병목 파악
- **쿼리 선택**: 어떤 쿼리를 벡터 부분으로 확장할 것인가
- **성능 기준선**: 벡터 검색 추가 전의 성능을 기준선으로 설정
- **새로운 병목**: 벡터 검색 추가 후 새로운 병목 출현 분석

구체적으로:

- Q1 (집계 쿼리)에 텍스트 검색 추가 → 집계기 여전히 병목?
- Q5 (조인 쿼리)에 벡터 필터 추가 → 조인 순서 최적화에 영향?
- 벡터 검색으로 인한 I/O, 메모리 변화 정량화
- 새로운 하이브리드 최적화 기법의 효과 측정

추가 제기 문제

1. **벡터 필터의 선택도 변화:** 벡터 검색 추가 시 중간 결과의 크기(선택도)가 어떻게 변할 것인가? 이것이 다운스트림 조인의 성능에 어떤 영향을 미칠 것인가?
2. **병목의 전환:** 원래 집계가 병목이던 쿼리에 벡터 검색을 추가할 때, 집계가 여전히 병목이 될 것인가, 아니면 벡터 검색으로 전환될 것인가?
3. **인덱싱 전략의 변화:** 벡터 필터와 관계형 필터를 모두 포함하는 쿼리에서 최적의 인덱싱 전략은? 벡터 인덱스를 먼저 사용할 것인가, 관계형 인덱스를 먼저 사용할 것인가?
4. **데이터 크기별 확장성:** Dreseler 논문의 분석이 벡터 데이터 추가 시에도 여전히 유효한가? 벡터 데이터의 메모리/I/O 특성이 다른 병목을 만들 것인가?
5. **다중 벡터 필드:** 실제 응용에서는 여러 텍스트 필드(제품 설명, 고객 리뷰, 태그 등)가 있을 때, 어떤 필드를 먼저 임베딩할 것인가? 성능 영향은?
6. **정적 vs 동적 최적화:** TPC-H의 쿼리는 정적이지만, 실제로는 벡터 검색 정확도가 동적으로 변할 수 있다. 이것이 최적 실행 계획 선택에 어떻게 영향을 미칠 것인가?